PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NOSQL

PROJECT REPORT

TOPIC – STUDENT MANAGEMENT SYSTEM USING MONGODB



SUBMITTED TO

Mr. Harendra Singh

SUBMITTED BY

HIMANSHU GUPTA

BCA (DS & AI)

ROLL NO. – 1250258221(69)

SCHOOL OF COMPUTER APPLICATIONS

BABU BANARASI DAS UNIVERSITY, LUCKNOW

ACADEMIC SESSION 2026–2027

INDEX

SR. NO.	SECTION	PAGE
1	Acknowledgement	3
2	Abstract	4
3	Introduction	5
4	Objectives	5
5	Methodology	6
6	Implementation Details	7
7	MongoDB Operations Demonstrated	8
8	Results	9
9	Conclusion	10
10	Future Scope	10
11	References	11

REPORT BASIS

This report adopts the structure and presentation style of the supplied sample project report, while its technical content is drawn from the supplied Student Management System Using MongoDB project.

ACKNOWLEDGEMENT

I would like to sincerely thank my teacher, Mr. Harendra Singh, for giving me the chance to undertake this NoSQL project, “Student Management System Using MongoDB”.

Working on this project strengthened my grasp of the fundamentals of NoSQL databases and showed me how MongoDB can be used to create, fetch, modify, sort and remove data. I also became familiar with the use of query operators such as greater than, less than, equal to, AND, and OR in MongoDB.

I am also grateful to my teacher for the guidance and support extended to me until this project was completed.

I further acknowledge the learning resources and tools used while implementing the project, which allowed me to practise MongoDB queries and understand document-based data management.

—HIMANSHU GUPTA

ABSTRACT

This report describes a Student Management System built with MongoDB, a document-oriented NoSQL database. The project shows how student information can be stored as documents and handled through MongoDB queries and operations.

The system relies on a Students database and a students collection holding 50 student records. Every record carries fields such as roll number, name, course, age, and marks. The project covers creating the database and collection, inserting, retrieving, filtering, updating, deleting, sorting, limiting, searching by course, and counting documents.

The comparison operators $eq, $gt, $lt, $gte, and $lte and the logical operators $and and $or are also demonstrated. As a result, the project gives practical exposure to the core NoSQL database operations in MongoDB.

KEY FEATURES

- Storage of student data as documents in MongoDB.
- Creation of a database and a collection.
- Insertion of 50 student documents.
- CRUD operations: create, read, update and delete.
- Comparison and logical query operators.
- Sorting, limiting, searching by course and counting.
- Hands-on execution and observation of MongoDB query outputs.

INTRODUCTION

MongoDB is a NoSQL database management system that keeps information in the form of documents. Rather than arranging data strictly in fixed rows and columns, MongoDB holds records as flexible documents, which makes it well suited to applications that need to store and query data in a document-oriented structure.

This project builds a Student Management System with MongoDB. The database holds details of 50 students, including roll number, name, course, age and marks. A series of MongoDB operations is carried out on these records to show practical database management.

The supplied project also shows how records can be filtered with comparison operators, combined with logical operators, changed with update operations, removed with delete operations, ordered with sort, restricted with limit, searched by course and counted.

OBJECTIVES

- To understand what NoSQL databases are.
- To become familiar with the core features and the document model of MongoDB.
- To create a database and a collection in MongoDB.
- To insert and manage 50 student records.
- To carry out CRUD operations on student documents.
- To apply comparison operators such as $eq, $gt, $lt, $gte and $lte.
- To carry out logical AND and OR operations.
- To sort, limit, search and count documents.

PROJECT DATA MODEL

FIELD	EXAMPLE	PURPOSE
rollno	101	Unique roll number of the student
name	Aarav Sharma	Name of the student
course	Computer Science	Course or program of the student
age	24	Age of the student
marks	65	Marks scored by the student

METHODOLOGY

The project follows a practical sequence of MongoDB operations. First the database is selected, then the collection is created and the student dataset inserted. The stored records are afterwards queried and manipulated with MongoDB operators.

STEP	ACTIVITY	DESCRIPTION
1	Create Database	Create or switch to the Students database using use Students.
2	Create Collection	Create a students collection inside the Students database.
3	Insert Records	Insert 50 student documents holding roll number, name, course, age and marks.
4	Retrieve Data	Display the stored student documents using find().
5	Filter Data	Apply comparison operators to fetch records based on age.
6	Update Data	Use updateOne() and updateMany() to modify documents.
7	Delete Data	Use deleteOne() and deleteMany() to remove records.
8	Logical Queries	Use $and and $or to join conditions.
9	Organize Results	Use sort() and limit() to control the query results.
10	Search and Count	Search by course and count the documents in the collection.

BASIC DATABASE SETUP

```
use Students
```

This creates the Students database, or switches to it.

```
db.createCollection("students")
```

This creates the students collection.

IMPLEMENTATION DETAILS

The implementation is based on the MongoDB shell/query syntax used in the supplied project. The student dataset holds 50 documents with roll number, name, course, age and marks.

INSERTING STUDENT RECORDS

```
db.students.insertMany([
 {rollno:101, name:"Aarav Sharma", course:"Computer Science", age:24, marks:65},
{rollno:102, name:"Priya Verma", course:"Electronics", age:26, marks:72},
{rollno:103, name:"Rahul Singh", course:"Mechanical", age:25, marks:68}, ...
{rollno:150, name:"Jyoti Verma", course:"Mechanical", age:25, marks:71}

])
```

The supplied project records that all 50 student documents were inserted successfully.

DISPLAYING RECORDS

```
db.students.find()
```

The find() operation fetches documents from the students collection. The supplied project displays the stored records and notes that the shell shows documents in batches.

COMPARISON OPERATORS

OPERATOR	EXAMPLE QUERY	PURPOSE
$eq	`db.students.find({ age: { $eq: 25 } })`	Age is exactly 25
$gt	`db.students.find({ age: { $gt: 30 } })`	Age above 30
$lt	`db.students.find({ age: { $lt: 25 } })`	Age below 25
$gte	`db.students.find({ age: { $gte: 30 } })`	Age 30 or more
$lte	`db.students.find({ age: { $lte: 25 } })`	Age 25 or less

MONGODB OPERATIONS DEMONSTRATED

UPDATE ONE

```
db.students.updateOne({ rollno: 102 }, { $inc: { age: 1 } })
```

Raises Priya Verma's age by one year. The supplied output shows matchedCount: 1 and modifiedCount: 1.

UPDATE MANY

```
db.students.updateMany({ course: "Computer Science" }, { $inc: { marks: 5 } })
```

Adds 5 marks for Computer Science students. The supplied output shows 14 documents matched and modified.

DELETE ONE

```
db.students.deleteOne({ rollno: 101 })
```

Removes the single document with roll number 101.

DELETE MANY

```
db.students.deleteMany({ course: "Mechanical" })
```

Removes all Mechanical-course documents; the supplied output shows 11 documents deleted.

AND

```
db.students.find({ $and: [ { age: { $gte: 25 } }, { course: "Electronics" } ] })
```

Returns students who are at least 25 years old and belong to Electronics.

OR

```
db.students.find({ $or: [ { age: { $lte: 25 } }, { course: "Electrical" } ] })
```

Returns students aged 25 or less, or students in Electrical.

SORT

```
db.students.find().sort({ age: 1 }).limit(5)
```

Sorts records by age in ascending order; the project also shows descending order using -1.

LIMIT

```
db.students.find().limit(3)
```

Shows only the first three remaining documents.

SEARCH BY COURSE

```
db.students.find({ course: "Electronics" })
```

Finds the students whose course is Electronics.

COUNT

```
db.students.countDocuments({})
```

Counts the documents currently in the collection; the supplied project reports a final result of 38.

RESULTS

The project successfully demonstrated the core NoSQL operations needed to manage a student dataset in MongoDB. The original dataset held 50 student documents, and queries were run to retrieve, filter, modify, delete and organize the records.

OPERATION	RESULT	OBSERVATION
Initial records inserted	50	50 student documents were inserted successfully.
updateOne()	1 matched / 1 modified	Age of roll no. 102 raised by one.
updateMany()	14 matched / 14 modified	Marks raised by 5 for Computer Science students.
deleteOne()	1 deleted	Roll no. 101 removed.
deleteMany()	11 deleted	Mechanical-course documents removed.
Final count	38	Count reported by the supplied project after the deletion operations.

LEARNING OUTCOMES

- Understanding of the MongoDB document model and the NoSQL approach.
- Practical use of the commands for creating databases and collections.
- Hands-on experience with CRUD operations.
- Understanding of comparison and logical operators.
- Ability to sort, limit, search and count MongoDB documents.
- Understanding of how query results change after update and delete operations.

PROJECT RESULT

The supplied project shows a complete practical workflow for managing a student dataset in MongoDB, running from database creation and insertion through querying, updating, deletion and the final document count.

CONCLUSION

This project gave hands-on experience with NoSQL databases through MongoDB. A Student Management System holding student details was built, and a variety of MongoDB operations were carried out on the data.

Through this work, I learned to insert, fetch, update and delete documents. I also learned to sort data in ascending and descending order, how comparison operators like $eq, $gt, $lt, $gte and $lte function, and how logical operators like $and and $or can be used to join conditions.

All in all, this project showed me how effectively MongoDB can manage and query student data.

FUTURE SCOPE

- Build a web-based student management interface connected to MongoDB.
- Introduce authentication and role-based access for administrators, teachers and students.
- Include attendance, subject-wise marks and semester-wise performance modules.
- Provide dashboards and charts for analysing student performance.
- Host the database on MongoDB Atlas and the application on a cloud platform.
- Apply validation, indexing and more advanced aggregation queries for larger datasets.

REFERENCES

- MongoDB Documentation – MongoDB Query and CRUD concepts.
- MongoDB documentation on comparison and logical query operators.
- MongoDB documentation on update, delete, sort, limit and count operations.

THANK YOU!

Student Management System Using MongoDB

HIMANSHU GUPTA

BCA (DS & AI) | Roll No. 1250258221(69)

Babu Banarasi Das University, Lucknow